

## 4 Analyse der NEORV32 CPU

Das Ziel dieses Kapitels ist die Analyse des bestehenden NEORV32 Cores. Die wichtigsten Elemente werden beschrieben. Durch die Analyse wird Klarheit darüber geschaffen, wie die Wirkungsweise der NEORV32 CPU ist und wie das System erweitert werden muss um die Matrixmultiplikation zu ergänzen.

### 4.1 Architektur des NEORV32 Core

Der NEORV32 besitzt den in Abbildung 4.1 dargestellten Aufbau. Diese Abbildung stammt aus dem NEORV32 Datenblatt [15].

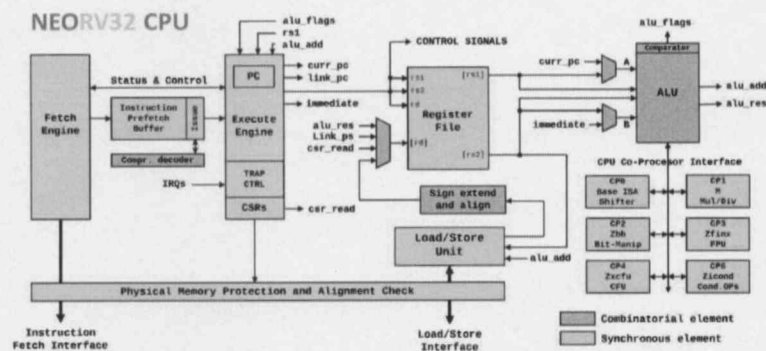


Abbildung 4.1: NEORV32 Architektur

Er ist im Groben aus einem Frontend und einem Backend aufgebaut. Das Frontend ermittelt die nächste Anweisung und das Backend führt diese Anweisung dann aus.

Beide Teile arbeiten in gewisser Weise unabhängig voneinander. Damit kann das Frontend bereits neue Instruktionen puffern während das Backend die aktuelle Anweisung abarbeitet. Im Falle von Sprüngen kann sich der Puffer dann mit den neuen Anweisungen am Sprungziel füllen, während das Backend parallel arbeitet.

ohne Trennung

## 4.2 Die Fetch-Engine

Die Fetch-Engine steht am Anfang der Abarbeitung einer Anweisung. Sie ist für das Laden der Anweisung aus dem Speicher zuständig und lädt von der Adresse, die momentan im Program-Counter-Register steht. Die Fetch-Engine arbeitet unabhängig vom Rest der CPU. Sie implementiert intern eine State-Machine, die Anweisungen puffern kann und sie kann intern Anweisungen laden ohne, dass die Anweisungen akut vom Rest des Systems gebraucht wird.

Aus diesem Grund gibt es ein weiteres Signal, welches in Abbildung 4.2 als "debug\_valid\_i" sichtbar gemacht wurde und angibt, ob die geladene Instruktion gültig ist und verwendet werden sollte oder nicht.

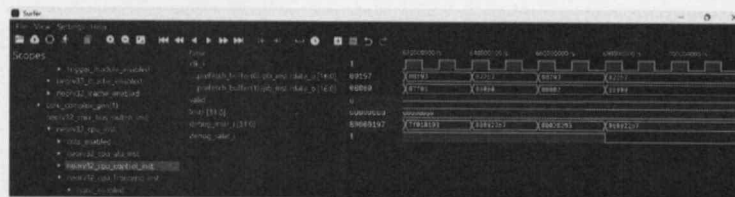


Abbildung 4.2: NEORV32 Gültige Instruktion

Dieses Wissen ist wichtig, wenn man das Verhalten der CPU anhand einer WaveForm-Ansicht diagnostizieren möchte. Dabei geht man oft von einem Assembler-Programm aus, das von der CPU ausgeführt wird. Wenn die Fetch-Engine kreuz und quer Instruktionen lädt, muss man wissen, ob die momentan anliegende Instruktion gültig ist oder nicht.

Es ist des Weiteren zu bemerken, dass die Fetch-Engine nur über einen Cache verfügt, wenn ein Cache konfiguriert wird. In der Standardkonfiguration ist kein Cache für Instruktionen konfiguriert, wie in der Abbildung 4.1 dargestellt. Die Fetch-Engine greift stattdessen direkt auf den Speicher zu.

## 4.3 CPU-Control und Execution-Engine

Bei einigen CPU-Designs für FPGAs findet man eine Kontrolllogik vor, die abhängig von der momentan geladenen Anweisung Steuersignale für den Datenpfad erzeugt. Der Begriff Datenpfad beschreibt zusammenfassend alle Hardware-Komponenten, die die Anweisungen wirklich berechnen. Beispiele für Komponenten im Datenpfad sind die Arithmetisch-Logische Einheit oder der Sign-Extender.

Die Kontrolllogik besteht oft aus kombinatorischer Logik und gibt damit direkt Signale aus, die ohne Speicherelemente direkt aus den Eingabesignalen berechnet werden. Ein Beispiel ist das Design aus [8]. Es ist in Abbildung 4.3 dargestellt. Allgemein wird die Kontrolllogik Einfluss auf den Datenpfad haben und Teile des Datenpfads an- und abschalten. Damit konfiguriert sich

eine CPU für jede Anwendung passend neu um diese Anweisung ausführen zu können.

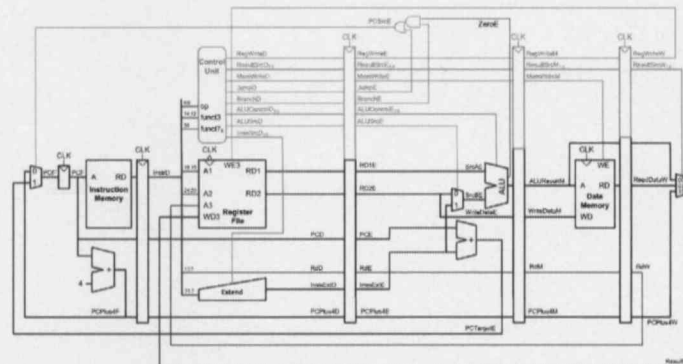


Abbildung 4.3: Design eines RISC-V CPU mit Pipeline

Sehr häufig kommen Pipelines zum Einsatz. Eine Instruktion durchläuft nacheinander die Stages der Pipeline. Die freien Stages können für andere Instruktionen verwendet werden. Damit erhöht sich der Durchsatz des Systems und die Auslastung der vorhandenen Hardware im Datenpfad erhöht sich.

Die NEORV32 CPU ist an dieser Stelle eine Ausnahme. Sie besitzt anstelle einer Pipeline eine State-Machine, die Execution-Engine genannt wird, um den Datenpfad zu kontrollieren.

Die Entität "CPU-Control" enthält die Kontrolllogik der NEORV32 CPU. Die CPU-Control enthält den Program Counter der auf die nächste Anweisung zeigt, die "Control and Status Register" (CSR) und Funktionen für Interrupts und Exceptions.

Die Execution-Engine ist im Inneren eines VHDL-Prozesses definiert. Die allermeisten Prozesse enthalten das Reset-Signal und das Clock-Signal in ihrer Sensitivitätsliste. Der Prozess für die Execution-Engine ist anders. Hier befindet sich kein Clock-Signal in der Sensitivitätsliste sondern alle Signale, die die Execution-Engine beeinflussen, müssen explizit aufgelistet werden! Dies wird wichtig, wenn die Execution-Engine später erweitert wird. Beispielsweise muss die Execution-Engine nachdem sie einen Request nach außen gibt, auch wieder auf dessen Response warten. Das Response-Signal muss in die Sensitivitätsliste aufgenommen werden.

Der Start-Zustand ist der EX\_RESTART state. Aus dem Reset geht die State-Machine in EX\_BRANCHED und erst dann in den EX\_DISPATCH Leerlaufzustand über. Der Umweg über EX\_BRANCHED erlaubt es auf ein Instruction Fetch zu warten.

Im Leerlaufzustand EX\_DISPATCH, wartet die Execution Engine bis ein Trap oder eine Exception eintritt oder bis das Frontend eine gültige Instruktion meldet, die dann abgearbeitet werden muss. Für normale Anweisungen wird der Zustand EX\_EXECUTE betreten.

Bindestile  
Len  
- Bindestile  
Lv,  
- Bindestile  
-  
-  
- Le,  
-  
- Le,  
-

f Le,

In EX\_EXECUTE wird der nächste Zustand abhängig von der Art der Anweisung gewählt. Das heißt EX\_EXECUTE ist ein Switch-State der in mehrere Äste der State-Machine verzweigen kann. Es gibt Zweige für ALU-Operationen, Sprünge und Speicherzugriffe (Load und Store).

Für länger laufende Befehle muss die Execution-Engine auf den Abschluss der Operation warten. Daher sind lange laufende Operationen jeweils durch ein Paar aus Request und Response Zuständen modelliert. Der Request-Zustand wird direkt in Richtung Response-Zustand verlassen. Der Response-Zustand wird erst in Richtung EX\_DISPATCH verlassen, wenn das eingehende Response-Signal aus der Sensitivitätsliste aktiviert worden ist. Im Response-Zustand werden die erzeugte Daten betrachtet und Signale zurückgesetzt.

Le,  
- Bindestrich  
↓  
-  
Le- Lt  
Bindestrich - 1P  
Bindestrich  
- Len

#### 4.4 Die Load-Store-Engine

Die State-Machine der Execution-Engine produziert eine Vielzahl an Signalen, die andere Entitäten des VHDL-Designs aktivieren und mit Eingabedaten versorgen. Ein wichtiger Teil des Designs ist die sogenannte Load-Store Unit. Die Load-Store Unit hat zur Aufgabe Bus-Requests zu erstellen und Bus-Responses zu verarbeiten.

Die Bus-Requests werden auf dem Bus innerhalb der NEORV32 CPU ausgeführt. Ein Teilnehmer am Bus ist z.B. der Datenspeicher. Mit Datenspeicher ist der Teil des Speichers gemeint, der die Werte der Variablen speichert, also konkret keinen Quellcode sondern eben Daten enthält. Der Datenspeicher wird über Load-Anweisungen gelesen und produziert damit Daten für die Register oder er wird durch Store-Anweisungen geschrieben. Die Aufgabe der Load-Store-Unit besteht darin die Signale der Execution-Engine in passende Bus-Requests für Load- oder Store Anweisungen zu übersetzen.

Dabei ist folgende Besonderheit zu beachten. Die Bus-Requests, die von der Load-Store-Unit erzeugt werden sind in der Datei neorv32\_top.vhd mit dem Data-Cache verbunden. Damit unterstützt die NEORV32 CPU eine Cache-Hierarchie für Daten. Anstatt direkt auf den Speicher zuzugreifen, greift die Load-Store-Unit immer zunächst in den Cache. Wenn ein Cache-Hit stattfindet, ergibt sich ein Geschwindigkeitszuwachs.

Bindestrich  
-  
- Le-  
- Le- Bindestrich  
-  
-  
Le,  
- Bindestrich  
-  
-  
- Le-  
1: Bindestrich  
-  
- Lt,  
-  
-  
-

#### 4.5 Zusammenfassung

In diesem Abschnitt wurde das Zielsystem, die NEORV32 CPU analysiert. Das Ziel war es die Komponenten kennenzulernen, die erweitert werden müssen um die Matrixmultiplikation hinzuzufügen.

Die NEORV32 CPU weicht von einem traditionellen Design mit Kontrolllogik,

1P Lt,

Datenpfad und Pipeline ab. Das spezielle Design wurde beschrieben. Es besteht aus einem Frontend und einem Backend. Die geplanten Änderungen finden im Backend statt.

Das Backend wurde anhand seiner relevanten Bestandteile Execution-Engine, Load-Store-Unit, Datenbus, Cache und RAM-Speicher beschrieben. Die einzelnen Bestandteile wurden besprochen.

Gerüstet mit dem Wissen um den inneren Aufbau des Backend kann nun im nächsten Kapitel definiert werden, welche neue Hardware für die Matrixmultiplikation ergänzt werden muss.

Ed,

## 5 Erweiterung der NEORV32-CPU

In diesem Kapitel werden die Erweiterungen an den Entitäten des vorgegebenen Designs beschrieben, die gemacht wurden, um die Matrix-Multiplikation zu ermöglichen.

Ln,

### 5.1 Die Matrix-Register

Die Matrixregister bestehen in der jetzigen Version aus 16 32-Bit Elementen und sind nicht für andere Datentypen konfigurierbar und nicht für größere Längen miteinander kombinierbar, wie es bei der RVV der Fall ist. Die Register sind wie folgt definiert.

Lt-

```
type ram_type is array (0 to MLEN-1)
  of std_ulogic_vector(XLEN-1 downto 0);
subtype matrix_ram_addr is integer range 0 to MLEN-1;

signal matrix_ram_0 : ram_type;
signal matrix_ram_1 : ram_type;
signal matrix_ram_2 : ram_type;
```

### 5.2 Die Instruktionen

Dieser Abschnitt beschreibt, welche Instruktionen hinzugefügt werden sollen und warum diese Instruktionen ausgewählt worden sind. Es wird beschrieben, wie man diese neuen Instruktionen im Übersetzungsprozess eines existierenden Compilers verwenden kann.

Um die Matrixmultiplikation durchzuführen sollen neue Instruktionen hinzugefügt werden, die es noch nicht bereits in anderen Extensions für RISC-V gibt. Die Instruktionen sollen das Strip-Mining für Matrizen ermöglichen und sich dabei an der RVV-Extension orientieren. Daher folgt zunächst ein kurzer Rückblick zur Vektor-Extension.

Bei der Diskussion des Strip-Mining bei der RVV-Extension wurde die Instruktion `set(i)vli` besprochen, mit der die Vektor-Engine für Datentypen und gewünschte Datenlängen konfiguriert wird. Des Weiteren wurden die Instruktionen `vle32.v`, `vadd.vv` und `vse32.v` im Rahmen eines Beispiels verwendet.

Ls  
IW



vle32.v lädt Daten aus dem Daten-Speicher in die Vector-Register (Load-Operation), vadd.vv ist die SIMD-Instruktion, die zwei Vektoren elementweise parallel miteinander addiert hat und vse32.v speichert die Daten schließlich in den Daten-Speicher zurück (Store-Operation)

LD-

LD.

Für die Matrix-Extension werden die Instruktion mle32, mmul32 und mse32 hinzugefügt. Dabei ist mle32 die Load-Operation, mmul32 die SIMD-Instruktion und mse32 die Store-Operation. Ein Äquivalent zur set(i)vi ist nicht vorgesehen, da die Matrix-Engine zum jetzigen Stand keine Konfigurationsmöglichkeiten anbietet.

keine Leerzeilen

Die NEORV32 CPU besitzt eine Menge an Beispielen, die sich im Software-Unterrordner befinden. Jedes Beispiel kann von Funktionen Gebrauch machen, die über die neorv32.h Header-Datei eingebunden werden können. Teil dieser Funktionen sind sogenannte Intrinsics. Ein Intrinsic ist eine Funktion, die in der Programmiersprache C geschrieben ist und die Programmiersprache C damit um Funktionalität erweitert, die thematisch zu einem anderen Themengebiet gehört.

Ein Beispiel dafür sind die Intrinsics the X-Window Systems [5]. Hier wird eine einfach zu verwendende Schnittstelle zur Softwareentwicklung mit dem X-Window System für die Programmiersprache erstellt.

Die Intrinsics des NEORV32 sind eine Erweiterung der Programmiersprache C um RISC-V Instruktionen aus C-Quellcode heraus auszugeben. Ein Beispiel für die Verwendung ist das folgende Testprogramm.

```

1 #include <neorv32.h>
2
3 uint32_t matrix_a[16]
4 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
5
6 uint32_t matrix_b[16]
7 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
8
9 int main() {
10
11     uint32_t *matrix_a_addr = &matrix_a[0];
12     uint32_t *matrix_b_addr = &matrix_b[0];
13
14     uint32_t matrix_unit_a = 0;
15     uint32_t matrix_unit_b = 0;
16
17     matrix_unit_a = CUSTOM_INSTR_L_TYPE(0x000, matrix_a_addr, 0b010
18                                     , 0b1011011);
19     matrix_unit_b = CUSTOM_INSTR_L_TYPE(0x001, matrix_b_addr, 0b010
20                                     , 0b1011011);
21
22     uint32_t matrix_mul = CUSTOM_INSTR_L_TYPE(0x000, matrix_a_addr,
23                                     0b100, 0b1011011);
24
25     matrix_unit_a = CUSTOM_INSTR_L_TYPE(0x000, matrix_a_addr, 0b001
26                                     , 0b1011011);
27     matrix_unit_b = CUSTOM_INSTR_L_TYPE(0x001, matrix_b_addr, 0b001
28                                     , 0b1011011);
29
30     for (int i = 0; i < 16; i++) {

```

```

26     neorv32_uart0_printf("After Matrix A %d\r\n", matrix_a_addr[i
    ]);
27 }
28 for (int i = 0; i < 16; i++) {
29     neorv32_uart0_printf("After Matrix B %d\r\n", matrix_b_addr[i
    ]);
30 }
31
32 return 0;
33 }

```

In der Zeile 17 ff wird ~~as~~ Intrinsic "CUSTOM\_INSTR\_I\_TYPE" für I-Type RISC-V Instruktionen verwendet. Aus jeder Zeile mit einem "CUSTOM\_INSTR\_I\_TYPE" Aufruf wird eine RISC-V Assembler Instruktion generiert.

La

Die Verwendung ist schwer zu verstehen, daher wird jetzt beschrieben, wie die Parameter zum Intrinsic zu interpretieren sind. Der letzte Parameter ist der Op-Code. Für die Matrix-Extension wurde in dieser Arbeit der Op-Code 0b1011011 gewählt. Innerhalb dieses Opcodes werden dann durch den vorletzten Parameter die drei Anweisungen mle32, mmul32 und mse32 kodiert.

Es ist noch abschließend zu erklären, weshalb Intrinsics verwendet werden müssen. Der grundsätzliche Ablauf bei der Verwendung der Programmiersprache C ist, dass ein Compiler den Quellcode der höheren Programmiersprache in Eigenregie in Assembler-Code übersetzt. Dabei wählt er die Anweisungen selbst aus. Optimierende Compiler führen diese Aufgabe in vielen Fällen besser aus, als es ein Mensch könnte.

Im Fall dieser Arbeit ist dieses Vorgehen nicht möglich. Der C-Compiler kennt die neu definierten Instruktionen mle32, mmul32 und mse32 schlicht nicht und wird sie daher nur ausgeben, wenn man den Compiler explizit dazu anweist. Diese explizite Anweisung passiert durch die Intrinsics.

Intern sind die Intrinsics durch Inline Assembly implementiert. Das bedeutet, dass dem C-Compiler exakt vorgegeben wird, welchen inline assembly Code er auszugeben hat. Im Beispiel-Code werden die Instruktionen mle32, mmul32 und mse32 per Opcode generiert.

Im Zuge dieser Arbeit ist ein einfacher Compiler erstellt worden, der auch dazu verwendet werden kann, die neuen Instruktionen auszugeben. Dieser Compiler wurde verwendet, bis der Weg über die Intrinsics des NEORV32 entdeckt wurde. Die Intrinsics erlauben es State-of-the-Art Compiler wie den GCC zu verwenden. Dies ist sehr viel komfortabler als einen eigenen Compiler zu pflegen.

lt

### 5.3 Erweiterung der Execution-Engine

Bis zu diesem Punkt liegt ein Beispiel-Programm mit den neuen Matrix-Instruktionen vor, welches in die NEORV32 CPU geladen und ausgeführt



werden kann. Damit wurde der Software-Anteil der Erweiterung beschrieben. Es folgt nun eine Diskussion der Erweiterungen der Hardware.

In diesem Abschnitt wird die Execution-Engine erweitert, so dass sie die neuen Instruktion tatsächlich ausführt, wenn sie von dem Frontend präsentiert werden.

Eine Übersicht aller Komponenten ist in Abbildung 5.1 zu sehen.

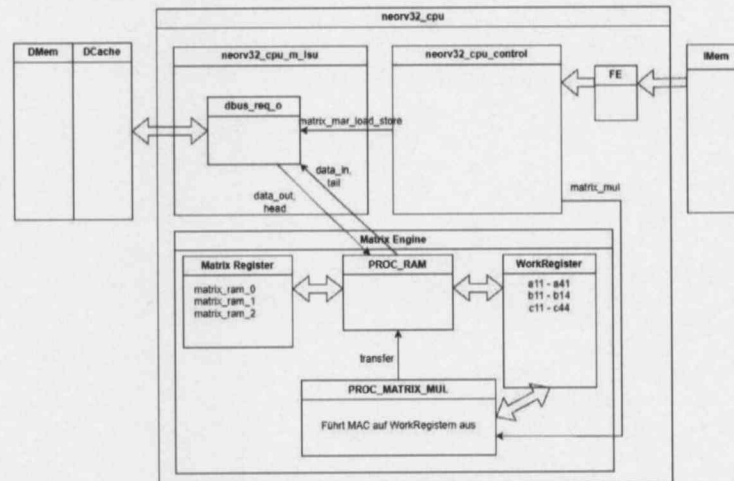


Abbildung 5.1: VHDL Entities

Die Instruktionen werden über das Frontend (FE) aus dem Anweisungsspeicher (IMem) geladen und an die Kontrolllogik weitergegeben. Die Daten der Matrix-Elemente kommen aus dem Datenspeicher (DMEM) und durchlaufen dabei den Daten-Cache (DCache).

Die Komponente neorv32\_cpu\_control gibt die Kontrollsignale matrix\_mar\_load\_store und matrix\_mul aus, wenn die Instruktionen mle32, mse32 bzw. mmul32 ausgeführt werden müssen.

Im Falle von mle32 (Load) werden die Signale data\_out und head verwendet um die 16 Matricelemente vom Bus-Request an die richtige Stelle in die Matrix-Register zu übertragen. PROC\_RAM übernimmt diese Koordinationsaufgabe.

Wird eine mmul32 Anweisung ausgeführt und damit das matrix\_mul signal von der Control-Logik gesetzt, dann beginnt der Prozess PROC\_MATRIX\_MUL mit der Multiplikation. Die Matrix-Register werden in die Arbeitsregister übertragen. Nach der Bearbeitung wird das Ergebnis aus den Arbeitsregistern wieder zurück in die Matrixregister übertragen.

Analog zu mle32 werden die Signale data\_in und tail für mse32 (Store) Anweisung verwendet um Daten aus den Matrixregistern zurück in den Datenspeicher und den Cache zu schreiben.

Im Folgenden werden die Änderungen im Detail beschrieben.

Als erstes wird der neue Matrix-Opcode aus der Erkennung illegaler Opcodes ausgeschlossen.

```
-- MATRIX EXTENSION
when opcode_matrix_c =>
    illegal_cmd <= '0';
```

Der Execution-Engine wird ein neues Signal "matrix\_operation\_wait\_i" in die Sensitivitätsliste eingefügt. Dadurch kann sie das Ende der Matrixberechnung erkennen und in den nächsten Zustand übergehen.

```
execute_engine_fsm_comb: process(
    ...
    matrix_operation_wait_i
    ...
)
```

Der ALU wird beigebracht, dass sie den Operand B als Immediate verwenden muss, sobald sie den Matrix-Extension Opcode sieht. Der Grund ist, dass die Load- und Store-Matrix-Anweisungen die Register-Id als Immediate Wert verwenden um zu ermitteln, welches Matrix-Register sie lesen und schreiben sollen. Damit muss die ALU diesen Immediate-Wert weitergeben, sonst ist er verloren und kann nicht mehr verwendet werden.

```
-- ALU operand B: is immediate? --
case opcode is
    -- add matrix opcode here because the mle32, mse32 instructions
    -- contain an immediate
    -- that needs to be added to the register encoded in the
    -- instruction
    when opcode_matrix_c | opcode_alui_c | opcode_lui_c ...
        ctrl_nxt.alu_opb_mux <= '1';
    when others =>
        ctrl_nxt.alu_opb_mux <= '0';
end case;
```

Im EX\_DISPATCH Zustand wird die Matrixmultiplikation gestoppt.

```
matrix_mul_o <= '0';
```

Der EX\_EXECUTE Zustand wird um die Behandlung der Matrix-Instruktionen erweitert.

```
-- Matrix Extension --
when opcode_matrix_c =>

case funct3_v is
    -- load from memory into matrix unit --
```

```

when funct3_mle32_c =>
  -- go to the specific states for the matrix extension.
  -- The states are inserted in order to load all the elements
    of a matrix in a loop
  exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

when funct3_mse32_c =>
  exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

when funct3_mmul32_c =>
  -- go to the state that processes matrix operations
  exe_engine_nxt.state <= EX_MATRIX_OPERATION_REQ;

when others => -- undefined
end case;

```

Für die drei Instruktionen mle32, mse32 und mmul32 werden neue Zustände in die Execution Engine eingefügt, so dass diese Zustände aus EX\_EXECUTE angesprungen werden können.

Ln-

Für die Matrix Load- und Store-Instruktionen wird der Zustand EX\_MATRIX\_MEM\_REQ hinzugefügt.

LX- Bindestric

```

-- trigger a loop of memory requests for the matrix
when EX_MATRIX_MEM_REQ =>

  -- tell the matrix unit to come online and process the request
  ctrl_nxt.lsu_m_en <= '1';

  if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
    exc_iaccess_c)) = '0') then

    -- memory request if no instruction exception

    -- tell the load store unit to process a request
    ctrl_nxt.lsu_req <= '1';

    -- forward the immediate from the instruction
    matrix_immediate_o <= immediate;

    -- rs1 (source register 1) carries the address to load from
    / store to
    matrix_mar_load_store_o <= rf_rs1_i;

    exe_engine_nxt.state <= EX_MATRIX_MEM_RSP;

  else
    -- On Error
    exe_engine_nxt.state <= EX_DISPATCH;
  end if;

```

Im Zustand EX\_MATRIX\_MEM\_RSP wird auf die Rückmeldung gewartet. Wenn die Rückmeldung anhand des Signals "lsu\_wait\_i" erfolgt ist, dann wird der geladene Wert über das write-back signal (wb) in die Register-File (RF) geschrieben. Dann wird die Matrix-Engine deaktiviert.

Bindestric

```

when EX_MATRIX_MEM_RSP => -- wait for memory response

```

```

if (lsu_wait_i = '0') then
    -- write-back to RF if read operation (won't happen in case
    -- of exception)
    ctrl_nxt.rf_wb_en    <= (not ctrl.lsu_rw) or ctrl.lsu_rvs or
        ctrl.lsu_rmw;

    -- tell the matrix unit to stay offline
    ctrl_nxt.lsu_m_en    <= '0';

    exe_engine_nxt.state <= EX_DISPATCH;
end if;

```

Für Matrixoperationen gibt es die beiden neuen Zustände EX\_MATRIX\_OPERATION\_REQ und EX\_MATRIX\_OPERATION\_RSP. Mit Matrix Operationen ist die Matrixmultiplikation gemeint. Da die Matrixmultiplikation nicht in einem einzigen Schritt ausgeführt ist, muss die Execution-Engine auf die Matrix-Engine warten. Daher ergibt sich die Notwendigkeit für einen Request- und einen Response-State.

cx

- Bindstrie

```

when EX_MATRIX_OPERATION_REQ => -- matrix add operation
    if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
        exc_iaccess_c)) = '0') then
        matrix_mul_o <= '0';

        case funct3_v is
            when funct3_mmul32_c =>
                matrix_mul_o <= '1';

            when others => -- undefined
        end case;

        matrix_immediate_o <= immediate;

        -- next state
        exe_engine_nxt.state <= EX_MATRIX_OPERATION_RSP;
    else
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;
when EX_MATRIX_OPERATION_RSP =>
    -- next state
    if (matrix_operation_wait_i = '0') then
        matrix_mul_o <= '0';

        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

```

Diese beiden Zustände sind sehr simpel. Im Request-State wird das Signal matrix\_mul\_o aktiviert und dann in den Response-State übergegangen. Im

- Bindstrie

Response-State wird auf das Signal `matrix_operation_wait_i` gewartet und dann wird die Matrixmultiplikation deaktiviert.

- Bindestrich

## 5.4 Erweiterung der Load-Store-Unit

Bindestrich

Der nächste Schritt ist die Erweiterung der Load-Store-Unit. Die Load-Store-Unit ist innerhalb der CPU-Entity definiert.

Bindestrich

Die Aufgabe besteht darin, Daten aus dem Datenspeicher in die neuen Matrix-Register zu laden. Hier wäre es möglich die Matrix-Engine als eigenen Bus-Teilnehmer an den Bus zu binden. Die Matrix-Engine hätte dann die Möglichkeit Daten aus dem Datenspeicher über den Bus zu laden.

Der Nachteil an diesem Ansatz ist, dass die Matrix-Engine und die Load-Store-Unit nicht mehr synchronisiert sind. Die Matrix-Engine wird Daten direkt aus dem RAM laden ohne auf den Cache zuzugreifen. Die Load-Store Unit hingegen verwendet den Cache weiterhin. Da die Load-Store-Unit verantwortlich für den Rest der CPU ist, greift die CPU dann insgesamt auf veraltete Daten zu.

Bindestrich

Um die Synchronisierung wieder herzustellen, müsste die Load-Store-Unit ihren Cache nach Ausführung einer Matrix- Load-Store-Operation invalidieren, da dieser veraltete Daten enthalten könnte. Daraufhin muss sich der Cache erst wieder aufwärmen und die Performance der CPU leidet folglich.

Aus diesem Grund wurde das Laden der Matrix-Engine direkt in die LSU integriert. Wenn eine Matrixoperation eine Matrix fertig berechnet hat und diese Daten dann wieder in den Datenspeicher übertragen werden, dann durchlaufen die Daten den Cache der LSU und wärmen den Cache damit für den Rest der CPU mit aktuellen Daten auf. Das bedeutet, dass der Rest der ausgeführten Anwendung dann direkt auf die gecachte Matrix zugreifen kann anstelle die Matrix erst wieder aus dem Datenspeicher laden zu müssen.

Damit werden Daten über die Load-Store-Unit geladen, damit die Daten im Cache landen. Die Load-Store-Unit erzeugt Bus-Anfragen wenn Sie Daten laden und speichern soll. Da ein Matrix-Register aus 16 Elementen besteht, werden beide Prozesse für Load und für Store erweitert um in einer Schleife 16 Elemente laden zu können.

Der Load-Prozess wird für Matrix-Load-Anweisungen um eine State-Machine erweitert. Der State `ML_IDLE` deaktiviert die Load-Operation. Der Zustand `ML_INIT` setzt den Zähler auf 0 und geht in den `ML_PROCESS` Status über. Im `ML_PROCESS` Status erfolgen 16 Load Operationen um das Matrix-Register zu füllen. Alle 16-Operation durchlaufen den Cache was positiv ist. Der `ML_INCREMENT` wird abwechselnd mit `ML_PROCESS` ausgeführt und zählt den Zähler nach oben.

Sind die 16 Operationen ausgeführt, geht die State-Machine zurück nach `ML_IDLE`.

"  
"  
1P  
"  
"  
"  
"  
Ld-  
Lnen Lc,

Bindestrich

Ein etwa gleichartiges Vorgehen wird auch für die Matrix-Store-Operation gewählt, welche 16 Elemente zurückschreiben muss.

Der neue PROC\_RAM Prozess produziert Steuersignale für den RAM-Speicher der Matrix-Engine. Abhängig von den Zählern, die in den Operationen Load und Store verändert werden, produziert der PROC\_RAM Prozess Signale, die die Load-Store-Unit verlassen und in der CPU mit dem RAM für die Matrix-Engine verbunden sind. Damit wird der RAM der Matrix-Engine gesteuert und gibt entweder Daten für ein Store zurück oder speichert Daten für eine Load-Anweisung. Die eigentlichen Daten kommen aus dem Daten-Speicher der CPU und durchlaufen den Cache der LSU.

Bindestriche

LM -

LM -  
Bindestriche

" "

## 5.5 Matrix-Engine

In die Datei "neorv32\_cpu.vhd" wird die Matrix-Engine eingebaut.

Die Matrix-Engine ist schematisch in Abbildung dargestellt.

L 5.2

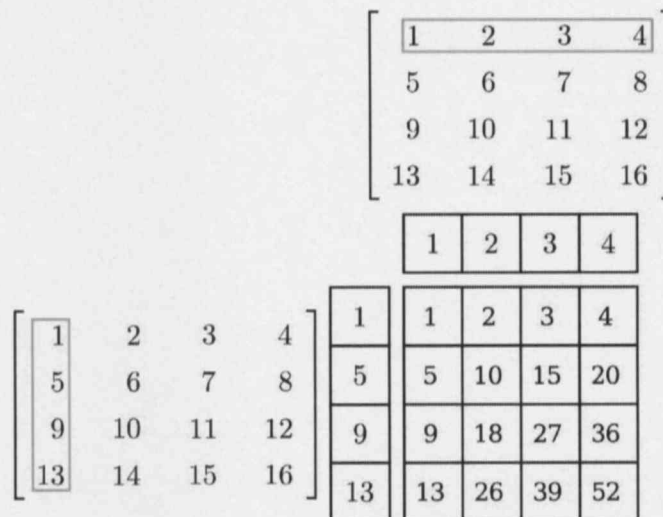


Abbildung 5.2: Matrix Engine

Auf der linken Seite und am oberen Bildrand befinden sich die beiden Matrixregister für die Matrizen A und B. Genau in der Mitte der Abbildung befindet sich die MAC-Einheit, in der die Outer-Products aufaddiert werden und in der sich das Ergebnis, also die Matrix C bildet. Zwischen den Matrix-Register und der MAC-Einheit befinden sich die Arbeitsregister.

LM

LMn

LMn,

Um die Matrix-Engine zu verwenden sind folgende Schritte notwendig:

1. Die Matrix Register A und B müssen aus dem Speicher bzw. dem Cache mit Matrix-Daten geladen werden.



2. Die MAC-Einheit muss mit Nullen gefüllt werden.
3. Die Arbeitsregister müssen mit einer Spalte der Matrix A und einer Zeile der Matrix B gefüllt werden.
4. Die MAC-Einheit wird verwendet um das Outer-Product zu berechnen und zu speichern.
5. Die vorhergehenden drei Schritte müssen insgesamt viermal wiederholt werden um zwei 4x4 Matrizen zu multiplizieren.
6. Der Inhalt der MAC-Einheit muss zurück in den Speicher und den Cache übertragen werden, damit er dann der Softwareapplikation zur Verfügung steht.

Die Engine besteht zunächst aus dem PROC\_RAM Prozess,

```

PROC_RAM : process(clk_i)
  variable l : line;
begin
  if (rstn_i = '0') then
    ...
  elsif rising_edge(clk_i) then
    if (transfer = '1') then
      -- store the c temporaries into the second matrix engine
      register
      matrix_ram_0(0) <= std_ulogic_vector(c11(31 downto 0));
      matrix_ram_0(1) <= std_ulogic_vector(c12(31 downto 0));
      ...
      matrix_ram_0(15) <= std_ulogic_vector(c44(31 downto 0));

      -- store the c temporaries into the second matrix engine
      register
      matrix_ram_1(0) <= std_ulogic_vector(c11(31 downto 0));
      matrix_ram_1(1) <= std_ulogic_vector(c12(31 downto 0));
      ...
      matrix_ram_1(15) <= std_ulogic_vector(c44(31 downto 0));
    end if;
    if (load = '1') then
      -- data_out is the output of the LoadStoreUnit (LSU) and it
      -- contains the RAM/cache value
      -- which is loaded into the MatrixEngine
      if (to_integer(unsigned(matrix_immediate)) = 0) then
        matrix_ram_0(head) <= data_out;
      else
        matrix_ram_1(head) <= data_out;
      end if;
    end if;
    if (store = '1') then
      if (to_integer(unsigned(matrix_immediate)) = 0) then
        data_in <= matrix_ram_0(tail);
      else
        data_in <= matrix_ram_1(tail);
      end if;
    end if;
    if (matrix_add = '1') then
      for i in 0 to 15 loop
        matrix_ram_0(i) <= std_ulogic_vector(unsigned(matrix_ram_0(i)
          )) + unsigned(matrix_immediate));
      end loop;
    end if;
  end if;
end process;

```

```
end if;
end process PROC_RAM;
```

Dieser Prozess hat die Aufgabe die Matrixregister zu speichern und zu laden und die Rechenergebnisse der Matrixmultiplikation in das Zielmatrixregister zu übertragen (transfer). Beim Speichern und Laden der Matrixregister werden die Signale head und tail verwendet, die von der LSU ausgegeben werden. Diese Signale beschreiben das Element, das geschrieben werden soll.

15f

Wenn die LSU Daten über den Bus aus dem RAM bzw. dem Cache erhält, dann wird gleichzeitig dazu das entsprechende Element im Register geschrieben oder gelesen. Die Synchronisation der LSU mit dem PROC\_RAM Prozess geschieht über die Signale head und tail.

Der zweite Teil der Matrix-Engine besteht aus der Berechnung des Outer-Product. Dazu sind bei 16 Elementen pro Matrixregister und damit mit 4x4 Matrizen vier Schritte notwendig. In den vier Schritten werden Zeilen aus der Matrix A und Spalten aus der Matrix B miteinander verrechnet. Mit vier Zeilen und vier Spalten müssen also vier Berechnungen pro Schritt ausgeführt werden.

Um die Berechnung auszuführen wird eine State-Machine verwendet. Die State-Machine durchläuft die Zustände MM\_RESET, MM\_INIT\_1/ und MM\_PROCESS\_1, bis MM\_INIT\_4, MM\_PROCESS\_4 und endet mit MM\_STORE, MM\_DONE.

- Bindestrich  
- 15f

Der MM\_RESET Zustand setzt die Matrixregister auf 0, wenn das matrix\_mul Signal auf 1 gesetzt wird. Solange matrix\_mul auf 0 steht, verbleibt der Wert der letzten durchgeführten Matrixmultiplikation in den Matrix Register stehen. Der Grund ist, dass man die Werte in den Register halten muss um sie in den RAM zurückschreiben zu können und dies einige CPU-Zyklen dauert. Das matrix\_mul Signal wird von der Kontrolllogik der CPU gesetzt, wenn ein mmul32 Befehl ausgeführt wird und steht ansonsten auf 0.

Lten LX- 15  
Lrn  
L1-  
L2-  
L1-  
L5-

Wenn das matrix\_mul Signal gesetzt wird, werden die Matrixregister mit 0 überschrieben und die State Machine geht in den nächsten Zustand über. Es folgen vier Schritte, die jeweils aus einem INIT und einem PROCESS Schritt bestehen. Alle vier Schritte sind äquivalent, es wird daher nur ein Schritt beschrieben.

Im INIT Schritt wird eine Zeile aus der A-Matrix und eine Spalte aus der B-Matrix in extra Signale geladen:

LT-

```
when MM_INIT_1 =>
  a11 <= unsigned(matrix_ram_0(0));
  a21 <= unsigned(matrix_ram_0(4));
  a31 <= unsigned(matrix_ram_0(8));
  a41 <= unsigned(matrix_ram_0(12));

  b11 <= unsigned(matrix_ram_1(0));
  b12 <= unsigned(matrix_ram_1(1));
  b13 <= unsigned(matrix_ram_1(2));
```

## 5 Erweiterung der NEORV32-CPU

```
b14 <= unsigned(matrix_ram_1(3));  
mat_mul_engine_state <= MM_PROCESS_1;
```

Im folgenden MM\_PROCESS Schritt erfolgt die sogenannte Multiply - Accumulate - Operation. Mit Multiply-Accumulate ist gemeint, dass eine Multiplikation als Kernoperation ausgeführt wird, aber das Ergebnis dieser Multiplikation nur ein Teilinkrement einer größeren Operation darstellt. In diesem Fall ist die größere Operation die Matrixmultiplikation, die sich aus vier addierten (kumulierten) Outer-Produkten zusammengesetzt. Die Outer-Produkte werden in den Akkumulator-Speichern c11 bis c44 aufgenommen.

LS  
Leereilen ram  
ld,  
n p

```
when MM_PROCESS_1 =>  
  c11 <= c11 + (a11 * b11);  
  c12 <= c12 + (a11 * b12);  
  c13 <= c13 + (a11 * b13);  
  c14 <= c14 + (a11 * b14);  
  
  c21 <= c21 + (a21 * b11);  
  c22 <= c22 + (a21 * b12);  
  c23 <= c23 + (a21 * b13);  
  c24 <= c24 + (a21 * b14);  
  
  c31 <= c31 + (a31 * b11);  
  c32 <= c32 + (a31 * b12);  
  c33 <= c33 + (a31 * b13);  
  c34 <= c34 + (a31 * b14);  
  
  c41 <= c41 + (a41 * b11);  
  c42 <= c42 + (a41 * b12);  
  c43 <= c43 + (a41 * b13);  
  c44 <= c44 + (a41 * b14);  
  
  -- next state  
  mat_mul_engine_state <= MM_INIT_2;
```

Nach vier MM\_INIT und vier MM\_PROCESS Zuständen ist die Matrixmultiplikation durchgeführt und wird schließlich im MM\_STORE Schritt in das Zielregister übertragen. Dazu wird das Signal "transfer" auf 1 gesetzt.

LS-  
LE-  
1 "

```
when MM_STORE =>  
  -- enable transfer in the RAM part  
  transfer <= '1';
```

Das "transfer"-Signal wird im PROC\_RAM Prozess gelesen und führt dort dazu, dass die MAC - Speicherzellen c11 bis c44 in das Ziel-Matrixregister kopiert werden. Dieser Sachverhalt wurde bereits weiter oben bei der Beschreibung des PROC\_RAM Prozesses beschrieben.

keine  
Leereilen

1 " - 15  
LM- M-

Es soll an dieser Stelle kurz bewertet werden, ob die Berechnung der vier Outer-Produkte auf diese Weise sinnvoll ist oder nicht. Als erstes ist diese Lösung funktional und diese Qualität ist sicherlich sehr wichtig. Ein VHDL —

Design zu erstellen, dass auch wirklich nach der Synthese auf einem FPGA funktioniert ist speziell für Anfänger immer ein großer Erfolg.

Als zweites ist der Quellcode vom Standpunkt der VHDL-Synthese aus zu betrachten. Aus Sicht einer höheren Programmiersprache ist die Implementierung nicht sehr sinnvoll, da man beispielsweise durch das Strassen-Verfahren einen Geschwindigkeitsvorteil erzielen kann. VHDL ist aber keine höhere Programmiersprache sondern eine Hardwarebeschreibungssprache. Dinge können in Hardware tatsächlich parallel ablaufen, wenn Elemente mehrfach synthetisiert werden.

Die 16 Multiplikationen pro MM\_PROCESS Zustand werden hoffentlich von der Toolchain auf Hardware-Blöcke innerhalb des FPGA abgebildet und laufen damit effizient und wirklich parallel ab. Wenn der FPGA mehrere Hardware-Blöcke für die Multiplikation hat, dann können diese Multiplikationen tatsächlich parallel ausgeführt werden.

Zusätzlich wäre es denkbar die vier MM\_INIT und MM\_PROCESS Schritte nicht sequentiell an eine State-Machine zu binden sondern sich über die parallele Ausführung der vier Schritte Gedanken zu machen. Dazu muss die Aufgabe gelöst sein eine Parallele Akkumulation der Teilergebnisse in die Speicher c11 bis c44 zu bewerkstelligen. Ein vierfaches, gleichzeitiges Schreiben in die Zellen ist ohne Synchronisation vermutlich nicht möglich. Aus diesem Grund wurde im ersten Schritt das sequentielle Vorgehen gewählt. Hier kann weiter optimiert werden.

Damit sind die wichtigsten Erweiterung des NEORV32 beschrieben.

## 5.6 Zusammenfassung

Die Umsetzung der Erweiterung wurde besprochen. Softwareseitig besteht die Umsetzung aus den neuen Anweisungen für den Umgang mit Matrizen. Es wurde beschrieben, wie die Anweisung ausgewählt worden sind und wie sie über einen Compiler verfügbar gemacht wurden.

Auf der Hardwareseite wurde beschrieben wie die Load-Store-Unit erweitert wird und wie die Matrix-Engine aussieht und mit Daten versorgt wird.

Im folgenden Kapitel soll die umgesetzte Erweiterung benutzt werden und Laufzeiten sollen gemessen werden um Entscheiden zu können, ob die Erweiterung tatsächlich einen Geschwindigkeitsvorteil bringt.

## 6 Auswertung und Evaluierung

In diesem Kapitel wird untersucht, ob der gewählte Ansatz zur Integration von Matrixoperationen in die NEORV32 CPU zielführend war. Zunächst wird die Funktionalität der Erweiterung geprüft. Das Prüfverfahren wird erläutert.

Danach wird untersucht, ob das erweiterte System eine Matrixmultiplikation schneller ausführen kann, als das unmodifizierte System. Um diesen Vergleich anzustellen wird die mathematische Vorschrift zur Matrixmultiplikation mit drei for-Schleifen auf zwei Matrizen ausgeführt. Die gleichen beiden Matrizen werden mit den hinzugefügten Anweisungen zur Matrixmultiplikation verrechnet.

Das Messverfahren wird beschrieben. Die Laufzeiten werden gemessen, ausgewertet und verglichen. Das Ergebnis dieses Vergleichs wird eingeordnet und diskutiert und es wird ein abschließendes Fazit gezogen.

### 6.1 Testdaten

Um eine Vergleichbarkeit der Verfahren und eine Überprüfung der Korrektheit zu erzielen, werden Testdaten mit erwartetem Ergebnis vorgegeben. In diesem Fall handelt es sich um zwei Matrizen A und B von denen das Ergebnis der Multiplikation bekannt ist und durch Matrix C dokumentiert wird.

L B,  
+ weils

Es wird eine 4x4 Matrixmultiplikation mit den beiden Matrizen A und B durchgeführt, für die das Ergebnis bekannt ist.

$$C = A * B \quad (6.1)$$

$$\begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

$$\begin{bmatrix} 5A & 64 & 6E & 78 \\ CA & E4 & FE & 118 \\ 13A & 164 & 18E & 1B8 \\ 1AA & 1E4 & 21E & 258 \end{bmatrix} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

Zur einfacheren Vergleichbarkeit der Matrix C mit der Waveform der Simulation wurde die Matrix C noch in hexadezimaler Schreibweise angegeben, da die Waveform Zahlen hexadezimal darstellt.

## 6.2 Messung und Vergleichbarkeit

Um Anweisungsfolgen miteinander zu vergleichen, muss zunächst eine Skala gewählt werden. Danach müssen die Anweisungsfolgen auf die Skala abgebildet werden. Die Abbildung muss dem Quellcode eine Zahl auf der Skala zuordnen. Als letztes muss für die Skala dann noch erwähnt werden, welcher Wert besser ist, also wie die Qualität von der Skala abzulesen ist.

Als Skala wird die Zahl der Clock-Ticks gewählt, die die CPU zur Ausführung der Anweisungsfolge benötigt. Die Abbildung erfolgt über ein Feature, dass die NEORV32 CPU liefert. Es handelt sich hier um Register, die die Clock-Ticks zählen. Die Qualität auf der Skala ist dadurch bestimmt, dass weniger Clock-Zyklen besser sind. Die CPU löst eine Aufgabe dann schneller.

Es wird nun das Zählen der Clock-Ticks beschrieben. In RISC-V Terminologie gibt es sogenannte CSR. CSR steht für Control- und Statusregister. Diese Register werden nicht bei der Abarbeitung von Anweisungen direkt verwendet. Sie können indirekt Einfluss auf die Arbeitsweise der CPU haben, da sie dessen Zustand enthalten. Über bestimmte Befehle lassen sich die CSR ändern und damit lässt sich der Zustand der CPU konfigurieren.

Ein Paar von CSR, dass bei der Zählung der Clock-Zyklen verwendet wird sind die Register CSR\_MCYCLE und CSR\_MCYCLEH. Das erste Register (CSR\_MCYCLE) speichert das Low-Word und das zweite Register (CSR\_MCYCLEH) speichert das High-Word des Clock-Cycle-Zählers.

Es werden wirkliche Clock-Cycles gezählt und keine Anweisungen. Das ist ein Vorteil, da damit eine wirkliche Messung erzielt werden kann. Wenn Anweisungen gezählt werden, ist nicht klar wieviel Zeit die CPU zur Bearbeitung der Anweisungen benötigt.

Ein Clock-Cycle beschreibt einen Ausschlag der externen Clock-Source. Die externe Clock-Source wird an das VHDL-Design als Input-Signal herangeführt. Dieses Signal wird in sehr vielen Prozessen des Designs verwendet um sequentielle, getaktete Logik voranzutreiben. Die NEORV32 CPU kann beispielsweise mit einer Taktrate von 100 Mhz betrieben werden. Damit ist die externe Clock-Source dann auf 100 MHz eingestellt. CSR\_MCYCLE und CSR\_MCYCLEH werden dann mit einer Taktrate von 100 Mhz inkrementiert.

Hier ist ein Beispiel für die Verwendung der Abbildung auf die Cycle-Skala:

```
1 // start timing
2 neorv32_cpu_csr_write(CSR_MCYCLE, 0);
3
```



```

4 for (i=0; i<(DATA_NUM/2); i++) {
5   v[0] = input_data[i*2+0];
6   v[1] = input_data[i*2+1];
7   xtea_sw_encipher(XTEA_ROUNDS, v, key);
8   cypher_data_sw[i*2+0] = v[0];
9   cypher_data_sw[i*2+1] = v[1];
10 }
11
12 // stop timing
13 uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
14
15 // -----
16 // Print timing results
17 // -----
18 neorv32_uart0_printf("\nExecution timing:\n");
19 neorv32_uart0_printf("ENC SW=%u cycles\n", time_enc_sw);

```

Im Beispiel wird nur das Low-Word und damit nur CSR\_MCYCLE verwendet. Die Erwartungshaltung ist offensichtlich, dass die gemessene Befehlsfolge berechnet ist, bevor der Zähler in das High-Word überläuft.

Da das CSR\_MCYCLE CSR-Register einen beliebigen Wert enthält, muss es zunächst in Zeile 2 auf 0 zurückgesetzt werden. Dann kann sein Wert nach Abarbeitung der Befehlsfolge in Zeile 13 ausgelesen und schließlich über eine serielle Schnittstelle ausgegeben werden.

Dieses Verfahren wird im Folgenden zur Bewertung der unterschiedlichen Matrixmultiplikationen verwendet. Es wird dabei inklusive der Übertragung der Daten von und zum CPU-RAM gemessen, da der Zugriff auf den CPU-RAM bei der Hardwarebeschleunigung berücksichtigt werden kann und damit ein Geschwindigkeitsvorteil erzielt werden kann.

Das Vorgehen ist das Folgende:

1. In Vivado wird das Design synthetisiert. Dabei wird der Bootloader als erstes Anwendungsprogramm direkt in das Design synthetisiert. Damit ist der Bootloader-Maschinencode direkt im internen Instruktion-Speicher der CPU enthalten und wird damit direkt beim Systemstart ausgeführt.
2. Der Bitstream wird über den in Vivado integrierten Hardware-Manager auf das FPGA-Board übertragen und dort ausgeführt. Der Bootloader ist jetzt über eine UART-Schnittstelle erreichbar.
3. Für jedes Testprogramm wird dieses Testprogramm mit dem GCC Compiler in der Version riscv-none-elf-gcc.exe (xPack GNU RISC-V Embedded GCC x86\_64) 14.2.0 übersetzt. Für die Übersetzung der Beispiele und Testprogramme sind im NEORV32 Projekt Skripte und auch das sogenannte "image\_gen" Werkzeug bereitgestellt. Mit "image\_gen" wird aus den Ausgaben des GCC eine Datei erstellt, die vom Bootloader verstanden wird.
4. Die Ausgabe des "image\_gen" Werkzeugs wird dem Bootloader über die UART-Schnittstelle zugeführt. Dazu wurde ein eigener Uploader er-

LE-

LER

Lns

Lmm 159

L2-

stellt. Die Firmware könnte aber auch über jeden beliebigen Terminal-Emulator übertragen werden.

5. Der Bootloader speichert das übertragene Programm im NEORV32 CPU und führt es aus, sobald der Uploader den Befehl zur Ausführung gibt.
6. Über die UART-Schnittstelle wird die gemessene Anzahl an Clock-Zyklen gelesen und notiert.
7. Die Ausgaben aller Testprogramme werden verglichen und bewertet.

### 6.3 Prüfung mit Matrix-Erweiterung

Für die Prüfung mit Hardwarebeschleunigung enthält das ausgeführte Anwendungsprogramm die `mle32`, `mmul32` und `mse32` Anweisungen als Intrinsics.

Die Matrix A und die Matrix B werden aus dem CPU-RAM in die Matrix-Engine geladen. Danach erfolgt die Multiplikation durch Aufruf der `mmul32` Anweisung, danach erfolgt das Speichern der Matrix A und Matrix B zurück in den CPU RAM. Matrix A enthält in der momentanen Implementierung das Ergebnis der Multiplikation.

```

1 #include <neorv32.h>
2
3 uint32_t matrix_a[16]
4 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
5
6 uint32_t matrix_b[16]
7 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
8
9 int main() {
10
11     uint32_t *matrix_a_addr = &matrix_a[0];
12     uint32_t *matrix_b_addr = &matrix_b[0];
13
14     uint32_t matrix_unit_a = 0;
15     uint32_t matrix_unit_b = 0;
16
17     neorv32_cpu_csr_write(CSR_MCYCLE, 0);
18
19     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b010
        , 0b1011011);
20     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b010
        , 0b1011011);
21
22     uint32_t matrix_mul = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr,
        0b100, 0b1011011);
23
24     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b001
        , 0b1011011);
25     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b001
        , 0b1011011);
26

```

```

27  uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
28
29  neorv32_uart0_printf("\nExecution timing:\n");
30  neorv32_uart0_printf("ENC SW = %u cycles\n", time_enc_sw);
31
32  return 0;
33 }

```

Zunächst wird über den WaveForm Viewer visuell geprüft, ob die korrekten Steuersignale ausgegeben werden. Danach wird geprüft, ob die Matrixmultiplikation in der Simulation mit den richtigen Daten abgearbeitet wird und ob die richtigen Werte in der Matrix C abgelegt werden.

Lm-  
Lie



Abbildung 6.1: Waveforms der Matrixmultiplikation

In Abbildung 6.1 ist zu sehen, dass sich über die Zeit hinweg neue Daten in den Registern a und b befinden. Es ist auch zu sehen, dass die Matrix-Engine die Werte aus den Registern a und b über das Outer-Produkt multipliziert und auf die Accumulator-Register c aufaddiert. Schließlich ist nach den vier Schritten zu sehen, dass das korrekte Ergebnis in den Accumulator Registern c steht und dort auch für einige Zeit stehen bleibt, so dass die Daten weiterverarbeitet werden können und nicht direkt gelöscht werden.

Lr-

Abschließend lässt sich sagen, dass die hardwarebeschleunigte Matrixmultiplikation mit dem erstellten Design wie geplant funktioniert.

## 6.4 Prüfung ohne Matrix-Erweiterung

Das Testprogramm für die Matrixmultiplikation ohne Hardwarebeschleunigung ist:

```

1  #include <stdio.h>
2
3  #define DIMENSION 4
4  #define ELEMENTS DIMENSION*DIMENSION
5
6  int standardMatrixMult(int* matrixA, int* matrixB, int* matrixC,
7  int rows, int columns) {
8
9      int counter = 0;
10
11      // over row of matrix B
12      for (int i = 0; i < rows; i++) {
13
14          // over column of matrix A
15          for (int j = 0; j < columns; j++) {
16
17              // fuse row and column together into a single cell of
18              matrix C
19              for (int k = 0; k < columns; k++) {

```

```

18
19         // int aldx = i * columns + k;
20         int temp_aldx = i * columns;
21         int aldx = temp_aldx + k;
22
23         // int bldx = k * columns + j;
24         int temp_bldx = k * columns;
25         int bldx = temp_bldx + j;
26
27         // int cldx = i * rows + j;
28         int temp_cldx = i * rows;
29         int cldx = temp_cldx + j;
30
31         int aVal = matrixA[aldx];
32         int bVal = matrixB[bldx];
33
34         int mult_temp = aVal * bVal;
35
36         int temp_2 = matrixC[cldx];
37         int temp_3 = temp_2 + mult_temp;
38         matrixC[cldx] = temp_3;
39     }
40 }
41
42 }
43
44 }
45
46     return 0;
47 }
48
49 int main()
50 {
51     neorv32_cpu_csr_write(CSR_MCYCLE, 0);
52
53     int matrixA[ELEMENTS] = {
54         1, 2, 3, 4,
55         5, 6, 7, 8,
56         9, 10, 11, 12,
57         13, 14, 15, 16
58     };
59     int matrixC[ELEMENTS] = {
60         0, 0, 0, 0,
61         0, 0, 0, 0,
62         0, 0, 0, 0,
63         0, 0, 0, 0
64     };
65     int matrixB[ELEMENTS] = {
66         1, 2, 3, 4,
67         5, 6, 7, 8,
68         9, 10, 11, 12,
69         13, 14, 15, 16
70     };
71
72     int a22 = standardMatrixMult(matrixA, matrixB, matrixC, 4, 4)
73         ;
74     uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
75
76     // -----
77     // Print timing results
78     // -----
79     neorv32_uart0_printf("\nExecution timing:\n");
80     neorv32_uart0_printf("ENC SW = %u cycles\n", time_enc_sw);
81

```

```
82     int resultPrettyPrintC = prettyPrintFormatMatrix(matrixC,
83                                     DIMENSION);
84     exit();
85     return 0;
86 }
```

Der C-Quellcode ist sehr außergewöhnlich gestaltet, einzig und alleine weil der selbsterstellte C-Compiler diesen Quellcode in dieser Form auch übersetzen können soll, falls weitere Tests durchgeführt werden sollten.

Le,  
L+S

### 6.5 Auswertung

Für die hardwarebeschleunigte Variante erfolgt folgende Ausgabe:

```
1 "\nBooting from 0x00000000...\r"
2 "\n\r"
3 "\nMatrix Multiplication Sample \r\r"
4 "\n\r"
5 "\nExecution timing:\r"
6 "\nENC SW=185 cycles\r"
```

Es werden 185 Zyklen notiert.

Für die nicht hardwarebeschleunigte Variante erfolgt folgende Ausgabe:

```
1 "\nBooting from 0x00000000...\r"
2 "\n\r"
3 "\nMatrix Multiplication Sample \r\r"
4 "\n\r"
5 "\nExecution timing:\r"
6 "\nENC SW=10385 cycles\r"
```

Es werden 10385 Zyklen notiert.

Laut gemessenen Werten und rein mathematisch betrachtet wird die Matrixmultiplikation zweier 4x4, vorzeichenloser Integer-Matrizen unter Ausführung der Matrix-Extension um einen Faktor von etwa 56 beschleunigt.

Die Messung vergleicht im Grunde zwei Testanwendungen. Die Testanwendung mit den Anweisungen aus der Matrix-Extension lässt sich nicht mehr stark optimieren. Das Vergleichsprogramm mit der herkömmlichen Multiplikation ließe sich durch eine bessere Implementierung sicherlich stark vereinfachen.

Optimiert man die gewählten Testprogramme weiter oder wählt man komplett andere Testprogramme, so ergibt sich ein anderes Verhältnis zwischen den Zyklen, die die Testprogramme verbrauchen. Es ist jedoch nicht abzusehen, dass sich das Verhältnis zu Gunsten einer herkömmlichen Implementierung

U

rung ohne Hardwarebeschleunigung wendet, auch wenn das Testprogramm stark optimiert wird.

Es lässt sich abschließend konstatieren, dass die Hardware-Beschleunigung einen positiven Einfluss auf die Laufzeit hat. Dies ist hauptsächlich auf die effiziente Übertragung der Elemente zwischen CPU-RAM und dem Prozess-RAM der Matrix-Engine zurückzuführen. Die 16 Datenelemente werden als Stream zwischen dem Speicher und der Matrix-Engine übertragen und dies geschieht ohne dass die Execution-Engine 16 einzelne Load- und Store Befehle ausführen muss. Der zusätzliche Aufwand wird damit gegenüber einer naiven Implementierung um ein Vielfaches reduziert. Der Geschwindigkeitszuwachs ist hauptsächlich auf diesen Effekt zurückzuführen.

Lt, Ls L



## 7 Zusammenfassung und Ausblick

Am Ende der Arbeit wird das Erreichte nochmals zusammengefasst um von diesem Standpunkt aus einen Blick in die Zukunft zu wagen.

Rückblickend lässt sich sagen, welche Entscheidungen zielführend und welche Entscheidungen hinderlich waren. Dies wird in der Zusammenfassung diskutiert.

Für die Zukunft werden im Ausblick Ideen angeführt, die während oder nach der Arbeit an der Lösung entstanden sind. Diese Gedanken sollen als Wegweiser für eine eventuelle Verbesserung oder Weiterführung der Arbeit dienen.

### 7.1 Zusammenfassung

Im Rahmen dieser Arbeit wurde die existierende NEORV32 CPU um Komponenten zur Matrix-Berechnung erweitert. Die Komponenten unterliegen starken Einschränkungen. Unter anderem können zum jetzigen Zeitpunkt lediglich 4x4 Matrizen mit vorzeichenlosen 32-Bit Zahlen berechnet werden. Zum termingerechten Abschluss dieser Arbeit war es notwendig solche Einschränkungen vorzunehmen. Für die Zukunft wäre es durch zusätzlichen Arbeitseinsatz möglich das Design um weitere Datentypen und Konvertierungen zu erweitern.

Die durchgeführten Änderungen orientieren sich an der Befehlsmenge der bereits existierenden RISC-V Vektor-Extension und greifen damit auch das Konzept des Strip-Mining auf. Strip-Mining erlaubt die Erstellung eines Hardware-Software-Co-Designs, welches in der Lage ist Matrixmultiplikationen größerer Matrizen durch Multiplikationen kleinerer Dimensionen zusammenzusetzen.

Um die Softwareentwicklung für Anwendungen und Testprogramme zu ermöglichen, wurden exzessive Anstrengungen unternommen, die etwa den gleichen Zeitaufwand wie die VHDL Entwicklung zur Folge hatten. Insbesondere wurde es durch einen Compiler ermöglicht die neuen Anweisungen auch unter Verwendung einer höheren Programmiersprache wie C in RISC-V Maschinencode zu übersetzen. Im Nachhinein wäre der schnellstmögliche Einsatz der NEORV32 Intrinsics ein sehr großer Vorteil gewesen. Leider wurde dieser Weg erst spät gelernt.

Es wurde ein Vergleich zweier Testprogramme auf dem NEORV32 Chip durchgeführt. Die Messungen zeigen einen Geschwindigkeitszuwachs um

1 E

7 ut

54

4+

4er

44-

4 V-

einen relevanten Faktor. Das Experiment ist geglückt und könnte in Zukunft durch die Erweiterung auf ein vollständiges Typsystem und weiter verbesserte Speicherzugriffe noch verbessert werden.

## 7.2 Ausblick

Am Ende dieser Arbeit drängt sich eine grundlegende Frage auf. In welchen Szenarien ist es sinnvoll Hardwarebeschleuniger in ein bestehendes RISC-V Design einzupflegen und ab wann sollte sich ein Unternehmen für die Erstellung eines komplett neuen Designs entscheiden. Diese Frage ist eng verknüpft mit der Frage ob der in dieser Arbeit gewählte Ansatz richtig oder falsch ist.

Im Zuge dieser Arbeit wurde ein bestehendes Design erweitert. Der Zugriffsweg auf den Speicher über die Load-Store-Unit und den am Bus angeschlossenen Speicher wurde wiederverwendet. Der Vorteil ist es sicherlich Kosten und Zeit sparen zu können, da existierende Komponenten wiederverwendet werden. Ein Nachteil ist eventuell, dass die wiederverwendeten Komponenten auf Anforderungen hin erstellt und optimiert worden sind, die in der Vergangenheit maßgebend waren, aber eventuell neue Erweiterungen nicht effizient unterstützen. Für diese Arbeit ist dieser Umstand nicht zu einem Problem geworden.

Die Alternative wäre es, ein grundlegend neues Design zu beginnen, welches dann die neuen Anforderungen besser abdeckt. Ein Beispiel für einen neuen Anforderung ist es beispielsweise mehr als einen Speicher-Zugriff parallel ausführen zu können. Die Hardware für die Outer-Produkt Berechnung könnte dann mehrmals synthetisiert werden. Jede Instanz der Outer-Produkt Berechnung könnte möglicherweise über ihren eigenen Zugriff auf den Speicher verfügen. Wenn der Speicher echt-parallel Daten über mehrere Ports gleichzeitig liefern kann, wird eine Matrix-Berechnung parallel statt sequenziell ausgeführt.

Ein Nachteil eines solch spezialisierten Designs ist es, dass es seine Allgemeinheit verliert. Ein RISC-V CPU soll eine allgemeine Rechenmaschine bleiben.

Es ist fallabhängig, welchen Weg ein Unternehmen wählen sollte. Ein finanzstarkes Unternehmen kann sich für die Erstellung von Anwendungsspezifischen Chips entscheiden, wie man es im Falle von Amazon und dem Trainium Chip sehen kann. Unternehmen wie NVIDIA erstellen Beschleunigerkarten, die für High-Performance Computing im Sinne von großer Parallelität eingesetzt werden können und damit ein größeres Feld abdecken. CPU Designs von Intel und AMD sind um Instruktionserweiterungen wie AVX2, AMX und FMA für die Vektor- und Matrixberechnung erweitert worden, wobei sie immer noch allgemeines Computing erlauben. In Zukunft wird RISC-V diesen Weg mit einer Matrix-Extension auch gehen und hat mit der RVV bereits begonnen.

1e LV-  
Le,

Lve  
Ln,

Ln, 1P  
Le, Lt-  
Lt-  
LII

1a  
1" LV-  
Ln,

Bitte beachten: immer Outer-Produkt

Wahrscheinlich ist es aus wirtschaftlicher Sicht notwendig zu wissen, was der Markt fordert und Hardware erstellen zu können, die die Anforderung des Marktes erfüllen. Aus akademischer Sicht ist es wichtig die Zusammenarbeit mit Firmen aus der Industrie zu suchen um innovative Lösungen für die zukünftigen Entwicklungen finden zu können.

Hi um Lh, Lg

Als nächste Schritte zur Fortführung dieser Arbeit ist es denkbar die Matrix-Extension intelligenter und damit performanter zu gestalten. Der Vergleich mit anderen Designs wie z.B. [13] und [16] führt zu neuen Erkenntnissen und einen Zuwachs an Erfahrung.

Ler 1p

Zusätzlich ist das System aus dieser Arbeit um weitere Datentypen zu ergänzen. Es gibt traditionelle Datentypen wie beispielsweise 8-, 16-, 32- und 64-bit ganzzahlige Datentypen mit und ohne Vorzeichen und auch 32- und 64-bit Gleitkommadatentypen. Dazu kommen neuartige Aufteilungen von Daten, wie sie für Large Language Models verwendet werden. Ein Beispiel ist hier NF4, was für normalized float mit 4-Bit steht. Sobald die Matrix-Extension mit unterschiedlichen Datentypen verwendet werden können soll ist über die Konvertierung zwischen den Datentypen nachzudenken. Es müssen ebenfalls dazu passende Instruktionen ergänzt werden. Die set(i)vl Instruktion aus der RVV-Extension benötigt dann ein Äquivalent in der Matrix-Extension um die Matrix-Extension für Datentypen konfigurierbar zu machen.

Len

L1,

Es ist zu prüfen, welche Ergebnisse das Lowering hervorgebracht hat. Wurden wirklich Hardware-Bausteine innerhalb des FPGA verwendet oder sind Anteile unnötigerweise durch Logikzellen synthetisiert worden? Hier lässt sich in Kleinstarbeit noch eine Optimierung durchführen, falls Logikzellen anstatt von vorhandener Hardware verwendet werden.

Ein weiterer Schritt ist es sicherlich eine Umgebung für die Validierung des Systems bereitzustellen. Aufgrund zeitlicher Beschränkungen wurde das Design nur visuell mit einem Waveform-Viewer geprüft. Die Sprache System-Verilog beispielsweise bietet weitreichendere Möglichkeiten zur Validierung.

Lei

## Quellenverzeichnis

- [1] AKENINE-MÖLLER, Tomas ; HAINES, Eric ; HOFFMAN, Naty: Real-Time Rendering 3rd Edition. Natick, MA, USA : A. K. Peters, Ltd., 2008. – 1045 S. – ISBN 987-1-56881-424-7
- [2] ALON AMID, et. a. Krste Asanovic A. Krste Asanovic: RISC-V "V" Vector Extension. <https://github.com/riscvarchive/riscv-v-spec/blob/master/v-spec.adoc>. Version: 2024
- [3] ARSALAN, Qizal: NEORV32 Hardware Accelerator (Arty A7) – Modified Core with CFU/CFS. [https://github.com/QizalA/neorv32\\_hardware\\_accelerator\\_arty\\_a7/2025](https://github.com/QizalA/neorv32_hardware_accelerator_arty_a7/2025)
- [4] ASANOVIC, Krste: RISC-V State of the Union. <https://riscv-europe.org/summit/2025/media/proceedings/2025-05-13-RISC-V-Summit-Europe-11h30-ASANOVI%C4%86-slides.pdf>. Version: 2025
- [5] DAVID FLANAGAN, Adrian N.: X Toolkit Intrinsics Reference Manual: for X11 Release 4 and Release 5. The Associates, 1990
- [6] FALK, Sigurd: Ein übersichtliches Schema für die Matrizenmultiplikation. Berlin : Wiley-VCH, Zeitschrift für Angewandte Mathematik und Mechanik (ZAMM), 1951
- [7] FUJIE FAN, David C.: Stream Computing RISC-V Matrix Extension Open Source Project Upgrades to Version 0.5, Supporting Vector+Matrix Implementation. <https://riscv.org/blog/stream-computing-risc-v-matrix-extension-open-source-project-upgrades-to-version-0-5-supporting-vectormatrix-implementation/>. Version: 2024
- [8] HARRIS, Sarah L. ; HARRIS, David: Digital Design and RISC-V Computer Architecture Textbook. In: 2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE) (2021), 1-5. <https://api.semanticscholar.org/CorpusID:246872104>
- [9] IBÁÑEZ, Roger F.: Introduction to the RISC-V Vector Extension. <https://eupilot.eu/wp-content/uploads/2022/11/RISC-V-VectorExtension-1-1.pdf>. Version: 2022
- [10] INTERNATIONAL, RISC-V: Integrated Matrix Extension. <https://riscv.atlassian.net/wiki/spaces/IMEX/pages/46071925/Charter>. Version: 2024

~

1. Version

Titel unter-  
streichen

Version ?

## Quellenverzeichnis

- [11] KILLIAN, Earl A.: VME Design Considerations. <https://riscv.atlassian.net/wiki/download/attachments/663781400/DesignConsiderationsForVME.pdf?api=v2>. Version: 2025
- [12] LARABEL, Michael: Qualcomm's Xqci RISC-V Extension Now Deemed Non-Experimental For LLVM 22. <https://www.phoronix.com/news/Qualcomm-Xqci-LLVM-Stable>. Version: 2025
- [13] MOREIRA, José E. ; BARTON, Kit ; BATTLE, Steven ; BERGNER, Peter ; BERT-RAN, Ramon ; BHAT, Puneeth ; CALDEIRA, Pedro ; EDELSON, David ; FOS-SUM, Gordon C. ; FREY, Brad ; IVANOVIC, Nemanja ; KERCHNER, Chip ; LIM, Vincent ; KAPOOR, Shakti ; FILHO, Tulio M. ; MUELLER, Silvia M. ; OLSSON, Brett ; SADASIVAM, Satish ; SALEIL, Baptiste ; SCHMIDT, Bill ; SRINIVASARAGHAVAN, Rajalakshmi ; SRIVATSAN, Shricharan ; THOMPSON, Brian W. ; WAGNER, Andreas ; WU, Nelson: A matrix math facility for Power ISA(TM) processors. In: CoRR abs/2104.03142 (2021). <https://arxiv.org/abs/2104.03142> Version: ?
- [14] NOLTING, Stephan: neorv32. <https://github.com/stnolting/neorv32>. Version: 2026
- [15] NOLTING, Stephan: The NEORV32 RISC-V Processor - Data Sheet. <https://stnolting.github.io/neorv32/>. Version: 2026
- [16] OSS, Stream C.: riscv-matrix-project. <https://github.com/riscv-stc/riscv-matrix-project>, 2025
- [17] THOMAS, Giles: Basic matrix maths for neural networks: the theory. <https://www.gilesthomas.com/2025/02/basic-neural-network-matrix-maths-part-1>. Version: 2025
- [18] TILO ARENS, FRANK HETTLICH, CHRISTIAN KARPfINGER, ULRICH KOCKEL-KORN, KLAUS LICHTENEGGER, HELLMUTH STACHEL: Mathematik. Springer Spektrum, Springer-Verlag, Berlin Heidelberg 2008, 2011, 2015, 2015
- [19] VLADIMIROV, Andrey: Optimization Techniques for the INTEL MIC Architecture. Part 2 OF 3: Strip-Mining for Vectorization. (2015), June. <https://colfaxresearch.com/download/711/> Version: ?
- [20] ZEE, Field G. ; GEIJN, Robert A.: BLIS: A Framework for Rapidly Instantiating BLAS Functionality. In: ACM Transactions on Mathematical Software 41 (2015), June, Nr. 3, 14:1–14:33. <https://doi.acm.org/10.1145/2764454> Version: ?

Titel unter -  
streichen

↓ Version

Titel unter -  
streichen

Titel unter -  
streichen